

Zhiwei Li

Full-Stack Engineer · AI Practitioner · Builder

lzwjava@gmail.com · github.com/lzwjava · lzwjava.github.io

About Me

- **11 years** in software engineering — backend, full-stack, AI
- Currently at **HSBC** — backend + AI-assisted development
- ~**1 billion tokens** consumed, top 6% Copilot premium usage at HSBC
- Training GPT-2 with **nanoGPT** on **H200 GPU**
- Previously: DBS Bank, HSBC PayMe, startup founder (30k users)
- **AI/LLM:** PyTorch, nanoGPT, Claude Code, OpenRouter, Copilot, llama.cpp
- **Backend:** Java, Spring Boot, Python | **Cloud:** AWS, Azure

AI Journey

- Completed **ML Specialization** & **Deep Learning Specialization** (Andrew Ng)
- Training GPT-2 124M with **nanoGPT** on H200 GPU
- ~**1 billion tokens** consumed via OpenRouter & other providers
- Built custom agents, prompt pipelines, automation tools
- Experimented with llama.cpp, embeddings, MMLU benchmark
- Developing autonomous workflows with **Claude Code** & **OpenClaw**

My Path to Understanding Transformers

- First read about **K, Q, V** mechanism around end of **2023** — didn't understand much
- Read *The Illustrated Transformer*, watched Karpathy, Umar Jamil, StatQuest
- Replicated neural network for handwritten digits **from scratch** — that's where real understanding began
- By mid-**2025**, transformers finally clicked — after **~2 years** of mulling
- Key insight: you don't get it the first time — the brain does its work over time

"If one can write it from scratch without copying any code, one understands very well."

Blog post: [Neural Network, Transformer and GPT](#)

From Neural Networks to Agents

How I Learn AI

GTY — Great Teacher Yin

Yin Wang (yinwang.org) — 140k followers on Weibo, independent thinker, CS researcher.

- Followed him since **2013** — he taught me how to think from first principles
- His professor: **Daniel P. Friedman** (Indiana University, born 1944)
- Yin Wang calls ML "**differentiable computing**" — it's just calculus
- His vision restoration method inspired my **3-year myopia experiment**
- Subscribed to his Substack for 2 years

"He not only taught me knowledge, but truly showed me how to think. He passed on the technique of catching fish from Daniel P. Friedman to me."

Blog post: [GTY - Great Teacher Yin](#)

Learning Philosophy

Inspired by **Yin Wang** and **Andrej Karpathy**

- **Print variables** to understand — not just read about it
- **Read real code** (nanoGPT, not just textbooks)
- **Train small models** yourself
- **Build systems**, not just theory
- Iterate like training an LLM — you don't get it right the first time

"Differentiable Computing"

"Machine learning is really useful, one might even say beautiful theory, because it is simply **calculus after a makeover**! It is the old and great theory of Newton, Leibniz, in a simpler, elegant and powerful form."

"There is no 'intelligence' in artificial intelligence, no 'neural' in neural network, no 'learning' in machine learning. What really works in this field is called '**calculus**'."

— **Yin Wang**

See through the buzzwords to the math underneath.

Learning Path Overview

Foundations (1–4): Neural net from scratch → PyTorch → language model

GPT (5–7): Transformers → train nanoGPT → scale up training

Agents (8–10): Chat model → coding agent → personal AI system

From math to GPT to AI system. That's the path.

Lesson 1 — Neural Networks from First Principles

Understand what a neural network really computes.

- Scalar, vector, matrix computation
- Forward propagation step by step
- Backpropagation with manual derivatives
- Activation functions, loss functions

Practice: Print every variable. Implement a 2-layer NN in pure Python. Train on MNIST.

After this, you understand every number inside a neural network.

Lesson 2 — From Neural Networks to Deep Learning

How deep learning actually trains.

- Gradient descent, learning rate, convergence
- Overfitting vs generalization
- Regularization, dropout, batch/mini-batch/SGD

Practice: Train a 3-layer classifier. Visualize the loss curve. Implement dropout manually.

Lesson 3 — PyTorch Minimal Framework

Read any PyTorch model with confidence.

- Tensor fundamentals, autograd
- `nn.Module` design, optimizer mechanics
- Dataset and DataLoader

Practice: Rebuild MLP in PyTorch. Train a CIFAR classifier. Inspect gradients.

Lesson 4 — Language Modeling Fundamentals

Understand what GPT predicts.

- Tokenization (BPE), n-gram models
- RNN/LSTM intuition
- Next token prediction, cross-entropy for language

Practice: Build a character-level language model. Train on tiny Shakespeare. Generate text.

GPT Architecture — The Big Picture

Reference: Andrej Karpathy — [Let's build GPT: from scratch, in code](#)

Input Text

- Tokenizer (text → token IDs)
- Token Embedding + Positional Encoding
- $N \times$ Transformer Blocks
 - (Self-Attention → Feed-Forward → LayerNorm)
- Linear → Softmax
 - Next Token Prediction

The entire model learns to predict: **given these tokens, what comes next?**

Tokenizer — Text to Numbers

Before the model sees anything, text must become numbers.

- **Character-level:** each character is a token — simple but slow
- **BPE (Byte Pair Encoding):** merge frequent character pairs iteratively
 - "learning" → ["learn", "ing"] → [4821, 278]
- GPT-2 uses ~**50,257** tokens; GPT-4 uses ~**100k**
- Tokenizer is trained **separately** before the model

"The pizza came out" → [464, 13293, 1625, 503]

Different tokenizers produce different token IDs — changing the tokenizer changes everything.

Word Embedding — Tokens to Vectors

Each token ID maps to a **learned vector** (e.g., 768 dimensions in GPT-2).

- token 464 ("The") → [0.12, -0.34, 0.56, ..., 0.08]
- These vectors are **not hand-crafted** — they're learned during training
- Similar words end up with similar vectors

Positional encoding is added so the model knows word order:

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d})$$

Without position info, "dog bites man" = "man bites dog" to the model.

Self-Attention — How Words Relate

| "The pizza came out of the oven and **it** tasted good."

How does the model know "it" refers to "pizza" and not "oven"?

Self-attention computes a similarity score between every pair of tokens:

1. Each token produces a **Query**, **Key**, and **Value** vector
2. Dot-product $\text{Query} \times \text{Key} = \text{attention score (how relevant?)}$
3. Softmax normalizes scores to weights (0 to 1)
4. Weighted sum of Values = output for that token

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Q, K, V — Intuition

Think of it like a **search engine** inside the model:

Component	Role	Analogy
Q (Query)	What am I looking for?	Your search query
K (Key)	What do I contain?	Page titles / descriptions
V (Value)	What's my actual content?	Page content returned

For the word "it":

- **Q** asks: "what noun am I referring to?"
- **K** of "pizza" answers: "I'm a noun, a food, the subject"
- **K** of "oven" answers: "I'm a noun, an appliance, inside a prepositional phrase"
- Attention score for "pizza" > "oven" → **V** of "pizza" contributes more

Multi-Head Attention & Transformer Block

Multi-head: run attention **multiple times in parallel** (e.g., 12 heads in GPT-2).

Each head can learn different relationships:

- Head 1: syntactic (subject-verb)
- Head 2: semantic (noun-pronoun)
- Head 3: positional (nearby words)

One Transformer Block:

Input

- Multi-Head Self-Attention + Residual Connection
- LayerNorm
- Feed-Forward Network (MLP) + Residual Connection
- LayerNorm
- Output

Training — Data, Loss, and Gradient Accumulation

Data: next-token prediction on massive text corpora.

- Input: [The, pizza, came] → Target: [pizza, came, out]
- Loss: **cross-entropy** between predicted and actual next token

Gradient accumulation simulates larger batch sizes on limited GPU memory:

- Instead of 1 batch of 64, do 8 mini-batches of 8
- Accumulate gradients, then update weights once
- Same math, fits in GPU memory

Mixed precision (fp16/bf16): halves memory, doubles speed.

Checkpointing: save model weights periodically to resume if training crashes.

Generation — From Trained Model to Text

After training, generation is **autoregressive** — one token at a time:

```
Prompt:  "The meaning of life is"  
Step 1:  → predict next token → "to"  
Step 2:  "The meaning of life is to" → "find"  
Step 3:  "The meaning of life is to find" → "purpose"  
...
```

Sampling strategies:

- **Greedy:** always pick the highest probability token
- **Temperature:** lower = more deterministic, higher = more creative
- **Top-k:** sample from top k candidates only
- **Top-p (nucleus):** sample from smallest set covering p% probability

nanoGPT implements all of these in ~300 lines of Python.

Lesson 6 — nanoGPT Deep Dive

The key lesson. Train your own GPT.

- nanoGPT training loop, model architecture
- Weight initialization, data pipeline, sampling

Practice: Train nanoGPT on a small dataset. Modify model size. Train on a Chinese corpus. Change the tokenizer.

Lesson 7 — GPT Training Engineering

Run real GPT training at scale.

- GPU memory optimization, gradient accumulation
- Mixed precision, checkpointing
- Multi-GPU training, dataset scaling

Practice: Train a 100M parameter model. Resume training. Fine-tune a model.

Lesson 9 — LLM Agents (Claude Code / OpenClaw)

Build an OpenClaw-style system.

- Tool calling, agent loop
- Planning and execution
- Memory systems, CLI agent design

Practice: Build a coding agent. Build a CLI automation agent. Build a multi-step reasoning agent.

Lesson 10 — Personal AI System (Capstone)

Build your own AI system.

- Local model deployment, RAG
- Long context memory, multi-agent systems

Final project: Personal coding AI, research assistant, CLI AI toolkit, or autonomous dev agent.

Open Source Highlights

Project	Commits	Tech
algorithm-solutions	2,466	Java
lzwjava.github.io	500	Jekyll, 9 languages
live-server	660	PHP
live-mobile-web	528	Vue / JS
code-review-server	275	PHP
code-review-web	302	Vue / JS
leachat-ios	556	Objective-C
leachat-android	412	Java

Beyond Engineering

- **Self-taught researcher** — 3 papers on natural vision restoration
- **431 blog posts** — translated to 9 languages with LLMs
- **Life hacker** — hundreds of small innovative practices
- **Traveler** — USA (twice), Hong Kong, Macao, half of China
- English learned from **60+ Filipino teachers** online
- **Crypto & US stock investor** since 2018 — ~20% annual returns
- Owns **hundreds of gadgets** — EMF meter, telescope, etc.

Reversing Myopia — 3 Years of Self-Experimentation

Inspired by **Todd Becker** and **Yin Wang**. 3 papers published. 3 years of data.

- Myopia worsens because we use **full-prescription glasses** for close-up work
- Core principle: "**Just barely clear**" — wear glasses $\sim 1.50\text{D}$ below full prescription
- Expect $\sim 0.25\text{D}$ **reduction every 3 months** with consistent practice
- Left eye: **350** $\rightarrow$ **250** (2022–2024), Right eye improving too
- Now using **multifocal progressive lenses** for daily use

The same first-principles thinking I apply to engineering — question the defaults, experiment, measure.

Blog post: [Vision Tips](#)

lzwjava.github.io — My Blog

A Jekyll blog with **515 original posts**, enhanced with AI-powered tools:

- **LLM Translation** — auto-translated to **9 languages** (EN, ZH, JA, FR, DE, etc.)
- **Google Cloud TTS** — audio versions of posts
- **XeLaTeX** — PDF generation for offline reading
- **EPUB** — ebook export for all post collections
- **GitHub Actions** — automated build, test, translate, deploy
- **MathJax** — renders math in technical posts
- **Night mode**, RSS feed, bilingual content

Plus **1,992 notes**, **323 Python scripts**, and **191 ML scripts**.

ww — Cross-Platform CLI Toolkit

github.com/lzwjava/ww — A Python CLI for developer productivity with LLM-powered helpers.

Group	Examples
Git	AI commit messages, squash, diff-tree, classify commits
Note	Create notes with git integration, obfuscate data
PDF	Markdown → PDF, batch pipeline, code → PDF
Image	Crop, remove background, compress, screenshots
Search	Multi-engine web search (Bing, DuckDuckGo, Ecosia)
Copilot	OAuth auth, model listing, chat with Copilot API
System	macOS/Linux utils, network scan, process management
Sync	Claude settings, bashrc, ssh configs

iclaw — AI Coding in Restricted Environments

github.com/lzwjava/iclaw — A minimal terminal REPL for AI-assisted coding via GitHub Copilot.

Built for **enterprise-constrained environments** — no browser extensions, no IDE plugins, just a Python CLI.

- **Multi-turn conversations** with Copilot (GPT-5.2 default)
- **Native tool calling** — web search, shell execution, file editing
- **Multiple search providers** — DuckDuckGo, Startpage, Bing, Tavily
- **GitHub OAuth** device flow authentication
- Slash commands: `/model`, `/search`, `/compact`, `/export`, `/copy`

When full-featured tools are blocked, iclaw gets the job done.

Let's Connect

Email: lwjava@gmail.com

GitHub: github.com/lwjava

Blog: lwjava.github.io

WeChat: lwjava

Thank You

lwjava@gmail.com · lwjava.github.io